Table des matières

Cours 07 — Traitement distribué avec Spark 1
Objectifs 1
1. Ce qu'une machine fait déjà 2
2. Le modèle de Spark : un pilote, des exécuteurs, des partitions 2
3. Paresseux : transformations et actions 3
4. Le brassage, le seul geste vraiment cher 3
5. Lire le plan d'exécution 4
6. Ce qu'un moteur distribué écrit 4
7. Trois pièges qui coûtent tout 5
8. Quand une machine ne suffit plus 6
9. Choisir un moteur, en pratique 6
À retenir 7
Exercices 7
Annexe A — Le détail des étapes du labo 8

Cours 07 — Traitement distribué avec Spark

Théorie · Module 7/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → ingestion → **lac** → entrepôt → restitution. Le Cours 06 a confié le pipeline à un orchestrateur. Ce cours-ci pose la question d'avant : ce que le travail demande comme machine. Il fait écrire le même traitement pour le moteur d'une machine employé depuis le Cours 02 et pour Spark, le moteur distribué que tout le monde nomme, sur les données de Boréal, et il mesure les deux.

Objectifs

- Nommer les pièces d'un moteur distribué : pilote, exécuteurs, tâches, partitions.
- Expliquer ce qu'est une partition, d'où vient leur nombre, et pourquoi il commande tout le reste.
- Distinguer une transformation d'une action, et dire ce que l'évaluation paresseuse change.

© 2026 iAlexMG. Tous droits réservés.

- Reconnaître un brassage dans un plan d'exécution, et dire ce qu'il coûte.
- Lire un plan d'exécution de bas en haut, des deux côtés, et y retrouver les filtres poussés.
- Reconnaître les trois pièges classiques : le brassage inutile, la partition déséquilibrée, et l'action qui ramène tout au pilote.
- Décider, sur des critères mesurables, si un traitement a besoin d'un moteur distribué.

---

1. Ce qu'une machine fait déjà

Avant de distribuer quoi que ce soit, on mesure ce qu'une seule machine fait. C'est la mesure que personne ne prend, et c'est la seule qui tranche.

Le travail, sur l'historien complet de Boréal	**Sur une machine**
lire 821 CSV, 2 407 796 lignes, et écrire le bronze en Parquet, un dossier par jour	2,7 s
agréger 2 405 366 points en 47 618 lignes, une par étiquette et par jour	0,19 s
dédoublonner 204 485 messages de télémétrie par une fonction de fenêtrage	0,33 s

- Un moteur d'une machine moderne lit le Parquet en colonnes, pousse les filtres jusqu'au fichier et occupe tous les cœurs sans qu'on le lui demande. Ce qui coûtait une grappe de machines (*cluster*) il y a dix ans tient aujourd'hui sur un portable.
- Ces chiffres sont le point de comparaison de tout le cours.

À retenir — la première mesure est celle du moteur que vous avez déjà. Un projet qui commence par installer un moteur distribué ne sait pas ce qu'il achète. La question n'est pas « Spark est-il rapide », elle est « qu'est-ce qui ne tient pas ici ».

2. Le modèle de Spark : un pilote, des exécuteurs, des partitions

Spark suppose plusieurs machines, même quand il n'en a qu'une : trois pièces, et une unité de travail.

La pièce	**Ce qu'elle fait**
le pilote (*driver*)	exécute votre programme, construit le plan, découpe le travail en tâches et les distribue ; il reçoit les résultats que vous demandez
les exécuteurs (*executors*)	des processus qui font le travail, chacun sur sa part des données

© 2026 iAlexMG. Tous droits réservés.

La pièce	Ce qu'elle fait
le gestionnaire de grappe	attribue les machines aux exécuteurs ; en local, c'est Spark lui-même
la partition	la part de données qu'une tâche traite, du début à la fin, sans regarder les autres

- Un traitement Spark est une suite de tâches, une par partition. Trop peu de partitions, et les cœurs attendent ; trop, et on paie l'ouverture de chacune ; inégales, et la plus grosse impose sa durée à tout le lot.
- Leur nombre se calcule : une partition vise 128 Mo (`spark.sql.files.maxPartitionBytes`), et ouvrir un fichier compte pour 4 Mo (`spark.sql.files.openCostInBytes`). Des fichiers de 145 Ko coûtent donc surtout leur ouverture, et Spark en met une trentaine par partition.
- `local[*]` loge le pilote et les exécuteurs dans un même processus, sur tous les cœurs : un vrai Spark, avec le même plan et le même brassage qu'en grappe, sans le réseau.

Analogie — la salle de tri. Le pilote est le chef de quai : il ne touche pas un colis, il découpe la cargaison en lots et dit qui prend quoi. Les exécuteurs sont les trieurs, une partition est un lot posé devant un trieur. Donner trente lots à douze trieurs fonctionne ; en donner un seul, énorme, fait travailler une personne pendant que onze regardent.

3. Paresseux : transformations et actions

Le code qui décrit le travail ne le fait pas.

Ce que vous écrivez	Ce qui se passe
`df = spark.read.csv(...)`	rien, sauf la lecture du schéma quand on ne le déclare pas
une transformation : `filter`, `select`, `groupBy`, `join`, `withColumn`	rien : le plan s'allonge d'un nœud
une action : `count`, `collect`, `show`, une écriture	tout le plan part d'un coup : lecture, filtre, agrégat, écriture

- Une erreur de chemin ou de type n'apparaît pas là où on l'a écrite, mais à l'action suivante : la pile d'appels montre l'action, pas la transformation fautive.
- Deux actions sur le même jeu refont le travail deux fois, à moins de demander à Spark de garder le résultat, par `cache()` ou `persist()`.

Piège — la mesure qui ne mesure pas ce qu'on croit. Chronométrer une ligne de transformation donne quelques millisecondes, et c'est faux. On chronomètre l'action, et on nomme dans le journal ce qu'elle a déclenché.

4. Le brassage, le seul geste vraiment cher

© 2026 iAlexMG. Tous droits réservés.

Un filtre, une projection, un calcul de colonne travaillent partition par partition : ces opérations sont dites étroites, et ne coûtent que la lecture. Le brassage (*shuffle*) commence dès qu'une ligne doit changer de partition : grouper, joindre, ordonner globalement. Spark écrit alors chaque partition sur disque, redistribue les morceaux par le réseau, et relit.

Ce qu'il faut savoir	Ce que ça donne
après un brassage, le nombre de partitions vient d'un réglage, `spark.sql.shuffle.partitions`	200 par défaut, un nombre choisi pour des grappes, absurde pour 47 618 lignes de résultat
l'exécution adaptative (`spark.sql.adaptive.enabled`, active depuis Spark 3) regarde la taille réelle des partitions après le brassage	elle replie les petites ensemble : les 200 partitions des agrégats du labo finissent en trois fichiers
un agrégat bien écrit agrège d'abord dans chaque partition, et ne brasse que les résultats partiels	deux `HashAggregate` encadrent un `Exchange` dans le plan

À retenir — ce qui coûte, ce n'est pas le calcul, c'est le déplacement. Quand un traitement Spark est lent, on cherche les `Exchange` dans son plan avant de toucher au code.

5. Lire le plan d'exécution

Le plan est la seule façon honnête de savoir ce qu'un moteur va faire. Les deux se lisent pareil : de bas en haut, la source d'abord, le résultat en dernier.

Ce que fait le nœud	Chez Spark	Chez DuckDB
lire le Parquet, avec le filtre poussé jusqu'au fichier	`FileScan parquet`, mention `PushedFilters`	`PARQUET_SCAN`, mention `Filters`
calculer la colonne du jour	`Project`	une projection
agréger dans chaque partition	le premier `HashAggregate`, en `partial_`	le `HASH_GROUP_BY`, seul
redistribuer les résultats partiels	`Exchange hashpartitioning`, avec son nombre de partitions	rien : un seul processus, une mémoire partagée
finir l'agrégat	le second `HashAggregate`	

- L'absence d'`Exchange` chez DuckDB n'est pas une optimisation : c'est la différence de nature entre les deux moteurs.
- La lecture des plans d'un moteur SQL, ses opérateurs et ses estimations de cardinalité sont au Cours 10 de la formation SQL avancé. Ici, il suffit de repérer la source, les filtres poussés et les échanges.

6. Ce qu'un moteur distribué écrit

© 2026 iAlexMG. Tous droits réservés.

Spark n'écrit pas un fichier, il écrit un dossier : chaque tâche écrit sa part, sous un nom qui porte le numéro de la partition, un identifiant unique et le codec de compression.

	Le moteur d'une machine	**Spark**
le bronze de l'historien	821 dossiers de jour, un `data_0.parquet` chacun, 12,7 Mo	821 dossiers de jour, un `part-00000-<identifiant>.c000.snappy.parquet` chacun, 18,8 Mo
la journée du 2025-10-28	22 277 octets	31 756 octets
les agrégats	un fichier, 497 Ko	trois fichiers, 557 Ko

- L'écart de taille vient du codec par défaut, pas du contenu : zstd au Cours 02, parce que le script le demande ; snappy chez Spark, sauf instruction contraire (`spark.sql.parquet.compression.codec`). Snappy décompresse un peu plus vite, zstd compresse nettement mieux.
- Un dossier de dix mille petits fichiers coûte cher à lire : chaque ouverture est une requête. C'est le défaut le plus commun des lacs tenus sans surveillance, et le Cours 10 montre comment on le répare.

7. Trois pièges qui coûtent tout

Le piège	**Ce qui se passe**	**La parade**
le brassage inutile	un `repartition` « pour bien faire » avant un agrégat qui brasse de toute façon double le travail	laisser l'agrégat brasser seul ; lire le plan
la partition déséquilibrée	redistribuer sur une clé dont les valeurs ne se valent pas donne des parts sans rapport, et la plus grosse fixe la durée du lot	regarder la répartition des clés avant de partitionner dessus
l'action qui ramène tout au pilote	`collect()` rapatrie toutes les lignes dans le processus qui ne devait que coordonner ; sur un gros jeu, une erreur de mémoire qui ne nomme pas le coupable	une action rend un résultat, pas un jeu de données : écrire, compter, agréger

- Les 58 étiquettes de l'usine sont l'exemple parfait du déséquilibre : le banc d'essai enregistre 242 202 points, l'état de marche du compresseur d'appoint 1 833, 132 fois moins.

Piège — `show()` n'est pas `collect()`, et c'est heureux. `show(20)` demande vingt lignes, et Spark s'arrête dès qu'il les a ; `collect()` demande tout. Le premier réflexe d'inspection est `show`.

© 2026 iAlexMG. Tous droits réservés.

8. Quand une machine ne suffit plus

Boréal n’a pas besoin de Spark. Reste à situer la frontière d’où le besoin viendrait.

Le signe	Ce qu’il veut dire
le jeu ne tient plus en mémoire, même par morceaux	une machine traite plus grand que sa mémoire, mais le disque devient le goulot
le temps imparti est dépassé alors que tous les cœurs travaillent	il n’y a plus rien à gagner sur une machine
la donnée vit déjà répartie, sur un stockage objet, et personne ne veut la rapatrier	le calcul se déplace vers la donnée, pas l’inverse
le traitement doit reprendre après la panne d’une machine, sans tout refaire	un moteur distribué rejoue les tâches perdues ; un processus unique recommence

- Une usine sur deux ans donne 2,4 millions de points. Vingt usines sur dix ans, à la même cadence, en donneraient environ 240 millions, une centaine de gigaoctets en Parquet : encore le terrain d’un bon moteur d’une machine, avec un partitionnement soigné.
- Le basculement arrive souvent pour d’autres raisons que le volume : plusieurs équipes qui lisent en même temps, une plateforme distribuée déjà en place dont s’écarter coûterait plus cher, un traitement qui doit tourner à côté de la donnée d’un lac d’entreprise.

À retenir — la bonne réponse dépend de ce qui est déjà là. Savoir écrire du PySpark reste utile quand une machine suffit : l’employeur a peut-être déjà une plateforme Spark, et le vocabulaire des partitions, des brassages et des plans sert partout.

9. Choisir un moteur, en pratique

L’ingénieur de données qui doit choisir un moteur pose toujours les mêmes questions, et y répond par une mesure.

Question	Ce qu’on vérifie	Chez Boréal (labo)
Que fait déjà une machine ?	la conversion et l’agrégat, chronométrés	2,7 s et 0,19 s (1.3, 2.2)
Le filtre descend-il jusqu’au fichier ?	le plan	`Filters: Quality=192` sous `PARQUET_SCAN` (2.5)
Les deux moteurs rendent-ils la même chose ?	les comptes, puis les lignes	2 407 796 et 47 618 des deux côtés (4.4, 5.3)
Où le travail se déplace-t-il ?	l’`Exchange` du plan	un seul, entre deux `HashAggregate` (5.2)
Deux résultats en flottants concordent-ils ?	une tolérance, pas une égalité	6 moyennes sur 47 618 à 0,001 (5.4)

© 2026 iAlexMG. Tous droits réservés.

Question	Ce qu'on vérifie	Chez Boréal (labo)
Le code évite-t-il les pièges ?	les trois pièges, mesurés	17,4 s contre 8,5 s ; 1 833 à 247 879 lignes par partition (7.2)
Faut-il un moteur distribué ?	le rapport des durées	de 17 à 55 fois plus lent en Spark, sur ce volume (8.1)
Le projet est-il revenu à son état ?	le bronze refait	550 087 lignes d'historien (8.4)

À retenir

- Un moteur distribué découpe les données en partitions et confie une tâche à chacune. Leur nombre vient des fichiers et de deux réglages, et il commande la durée de tout le traitement.
- Spark est paresseux : les transformations allongent un plan, seule une action l'exécute. Deux actions sur le même jeu refont le travail deux fois.
- Le brassage redistribue les lignes entre partitions : c'est le seul geste vraiment cher, et il sort de tout groupement, de toute jointure et de tout tri global.
- Un plan se lit de bas en haut : `FileScan` et ses filtres poussés, `Exchange` pour le brassage, deux `HashAggregate` autour de lui quand l'agrégat est bien écrit.
- Spark écrit un dossier de fichiers, en snappy par défaut : 18,8 Mo là où le zstd du Cours 02 en écrit 12,7, pour le même contenu.
- Les trois pièges coûteux sont le brassage inutile, la partition déséquilibrée et l'action qui ramène tout au pilote ; les trois se voient dans le plan ou dans les tailles de partition.
- Sur l'historien complet de Boréal, une machine fait en 2,7 s ce que Spark fait en 46 à 77 s, avec les mêmes résultats. Le distribué se justifie par ce qui ne tient pas, jamais par principe.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_07_DE.md`. Faites-les après l'étape 7 du labo et avant l'étape 8, qui efface les résultats des deux moteurs : plusieurs énoncés supposent que les durées sont sous vos yeux et que les deux jeux de résultats existent encore. Le labo déjà fini, l'en-tête des exercices dit quelles étapes refaire.

© 2026 iAlexMG. Tous droits réservés.

Annexe A — Le détail des étapes du labo

A.0 Avant de commencer : l'image de Spark

Notions : section 2, le modèle de Spark.

- Spark ne s'installe pas : il se tire une fois, comme image de conteneur, et il ne reste rien entre deux lancements. Docker Engine tourne dans l'Ubuntu de WSL 2 depuis le Cours 04 (`Data_Domain/00_Installation/Guide_installation.md`, étape 4).
- L'image officielle porte Spark 4.2.0, Scala 2.13, Java 21 et Python : 2,22 Go, téléchargés une seule fois. Une image n'est pas un conteneur, et rien ne démarre.
- La fenêtre 2 s'ouvre dans `Projet_Boreal`, et non dans `plateforme` comme au Cours 06 : le lanceur monte le dossier courant dans le conteneur. La pile d'Airflow reste arrêtée, pour laisser tous les cœurs à Spark.

A.1 Étape 1 : l'historien complet, et le bronze sur une machine

Labo : `Labo_07_DE.md`, étape 1.

Notions : section 1, ce qu'une machine fait déjà.

- 1.1. La commande du générateur porte deux options, et elles se cumulent :

L'option	Ce qu'elle décide	Sans elle
`--jusqua 2026-03`	les sources vont jusqu'à la fin de mars 2026 : les trois mois de 2026 du Cours 06 restent	les sources s'arrêtent au 31 décembre 2025
`--historien complet`	l'historien livre ses 821 jours, depuis la mise en service du SCADA, le 2024-01-01	l'historien ne livre que 182 jours, du 2025-10-01 au 2026-03-31, comme au Cours 06

Le générateur réécrit `sources/` en entier ; chez l'auteur, 143,4 s. L'historien pèse désormais 117 Mo de CSV, contre 8 Mo au Cours 06, et la télémétrie compte 974 jours, 204 485 messages de 7 644 véhicules, dont 2 943 doublons.

- 1.2. Ce qui dérive des sources ne suit pas tant qu'on ne relance pas l'étape qui le produit : le bronze gardait les 550 087 lignes d'hier. Le pipeline refait les deux bronzes en 29,8 s, et ses contrôles, qui recomptent les CSV et les Parquet des deux côtés, en prennent les trois quarts.
- 1.3. Le script seul isole la conversion : 2,7 s pour le `COPY` chez l'auteur, puis 21,9 s pour le contrôle. `.timer on` chronomètre chaque requête à part, alors que `duckdb -c "q1; q2"` chronométrerait le lot d'un bloc.

A.2 Étape 2 : les agrégats sur une machine

Labo : `Labo_07_DE.md`, étape 2.

© 2026 iAlexMG. Tous droits réservés.

Notions : section 1, la mesure étalon ; section 5, lire le plan.

- 2.1. Le dossier `comparaison/` vit dans `Projet_Boreal`, hors du dépôt : il ne sert qu'au labo, et s'efface en 8.5.
- 2.2. Deux dixièmes de seconde pour parcourir 2,4 millions de points et en rendre 47 618.
- 2.4. La somme des moyennes, 2 737 589,987, servira à la comparaison avec Spark en 5.2.
- 2.5. `EXPLAIN` montre le plan sans exécuter la requête. Le bloc `PARQUET_SCAN` ne remonte que deux colonnes, `TagName` et `Timestamp`, parce que la requête ne fait que compter et que le Parquet est rangé par colonnes ; `Quality` ne sert qu'au filtre, appliqué dans le fichier (`Filters: Quality=192`) ; `~254,674 rows` n'est qu'une estimation, tirée des statistiques du Parquet. Au-dessus, un `HASH_GROUP_BY` fait tout le regroupement, et rien ne se déplace.

A.3 Étape 3 : le bronze en PySpark

Labo : `Labo_07_DE.md`, étape 3.

Notions : section 2, les partitions ; section 3, la paresse.

- 3.2. Ce que décide chaque option du lanceur :

L'option	Ce qu'elle décide
`--rm`	le conteneur disparaît à la fin ; rien ne tourne entre deux lancements
`--user root`	Spark change les droits des dossiers qu'il crée, ce qu'un disque Windows monté refuse à tout autre utilisateur
`-v "$PWD":/projet` et `-w /projet`	le projet entre dans le conteneur, et devient son dossier courant : les chemins des scripts sont relatifs au projet
`--master "local[*]"`	un pilote et des exécuteurs dans le même processus, sur tous les cœurs

- 3.3. Le schéma se déclare, comme au Cours 02 : deviner coûte une lecture de plus, et se trompe. Rien ne lit un octet de données avant `historien.write` : les trois `print` du début interrogent le plan et la configuration, et sortent aussitôt ; seule l'écriture, une action, lit les fichiers.
- 3.4. Avec les retours chariot, bash lit `set -e` suivi d'un caractère invisible, puis coupe la commande `docker run` à la première ligne : l'image demandée devient ce caractère, d'où `invalid reference format`, et chaque ligne suivante part seule, en `command not found`. Le message de la ligne 10 sort brouillé : affiché, le retour chariot ramène le curseur au début de la ligne, et la fin du message écrit par-dessus son début.
- 3.5. `sed -i 's/\r$//'` retire le retour chariot en fin de chaque ligne, sur place. Le `.gitattributes` de 8.6 empêche Git de le remettre au prochain clone.

© 2026 iAlexMG. Tous droits réservés.

- 3.6. Les 821 fichiers donnent 27 partitions de lecture, et non 821 ni 12 : la trentaine de fichiers par partition de la section 2. Les 200 partitions de brassage ne serviront qu'en 5.2. Le `FileNotFoundException` vient de ce que Spark sonde le chemin littéral avant d'étendre le joker. Le résultat est juste au chiffre près ; la conversion prend de 46 à 77 s chez l'auteur, sur quatre lancements, et le lancement entier de 83 à 117 s, démarrage du conteneur compris. Le journal de Spark défile dans la même fenêtre, et les `WARN MemoryManager` disent seulement que les tâches qui écrivent en même temps se partagent la mémoire du conteneur.
- 3.7. La durée varie du simple au double d'un passage à l'autre : une durée distribuée se cite en fourchette, jamais en valeur unique.

A.4 Étape 4 : ce que les deux moteurs ont écrit

Labo : `Labo_07_DE.md`, étape 4.

Notions : section 6, ce qu'un moteur distribué écrit.

- 4.1 et 4.2. Le même jour, 22 277 octets en zstd, 31 756 en snappy. Le nom du fichier de Spark porte le numéro de la partition, un identifiant de tâche et le codec ; le `.crc` à côté est la somme de contrôle que Spark écrit pour chaque fichier.
- 4.3. 12 740 766 octets contre 18 765 646, pour exactement les mêmes lignes ; `du -h` les arrondit au mégaoctet supérieur. Sans `--apparent-size`, `du` compterait la place occupée par blocs de 4 Ko, et gonflerait les deux côtés.
- 4.4. Un dossier par jour des deux côtés, 821 chacun, et un seul fichier par dossier : chaque jour tient dans une partition, parce que chaque fichier source est un jour. Sur des données brassées, Spark écrirait autant de fichiers par dossier qu'il a de partitions.

A.5 Étape 5 : les agrégats en Spark

Labo : `Labo_07_DE.md`, étape 5.

Notions : section 4, le brassage ; section 5, lire le plan.

- 5.2. `explain` imprime le plan avant l'action. Le premier `HashAggregate` réduit 2,4 millions de points à quelques milliers de lignes par partition ; seul ce résultat partiel traverse l'`Exchange`. 10,6 s contre 0,19 s, chez l'auteur, pour les mêmes comptes, et une somme des moyennes à 2 737 589,989 contre 2 737 589,987.
- 5.3. Les comptes et les extrêmes sont exacts des deux côtés ; six moyennes sur 47 618 diffèrent.
- 5.4. Les six écarts valent tous 0,001 : une somme de flottants dépend de l'ordre des additions, et les deux moteurs ne partitionnent pas le travail pareil.

A.6 Étape 6 : la télémétrie dédoublonnée

Labo : `Labo_07_DE.md`, étape 6.

Notions : section 4, le brassage d'une fenêtre.

© 2026 iAlexMG. Tous droits réservés.

- 6.1. Le courtier MQTT livre « au moins une fois » : l'archive contient 2 943 messages en double. Les écarter demande de numéroter les messages qui partagent un identifiant, par une fonction de fenêtrage (`QUALIFY ROW_NUMBER() OVER (…) = 1`), et une fenêtre brasse tout. La fonction de fenêtrage elle-même est au Cours 04 de la formation SQL avancé.
- 6.2 et 6.4. Les mêmes 201 542 messages des deux côtés, sur les 204 485 reçus, dans 933 jours.
- 6.4. Le pire rapport du labo, 11,0 s contre 0,33 s, une trentaine de fois : tous les messages d'un même identifiant doivent se retrouver dans la même partition, et il y en a 201 542.

A.7 Étape 7 : les trois pièges

Labo : `Labo_07_DE.md`, étape 7.

Notions : section 7, les trois pièges.

- 7.2. Le `repartition` avant l'agrégat ajoute un brassage complet avant celui que l'agrégat allait faire : 17,4 s contre 8,5 s, pour le même résultat.
- 7.2. Redistribuer sur l'étiquette donne 37 partitions non vides sur 58 : deux étiquettes tombent parfois dans la même case de hachage, et d'autres cases restent vides. La plus grosse porte 247 879 lignes, la plus petite 1 833, et onze cœurs attendent le douzième.
- 7.2. Le `collect` d'une étiquette met 7,4 s pour 1 833 lignes, presque tout en balayage du bronze. Celui de tout le bronze ramène 2 407 796 lignes en 19,8 s et passe, parce que le jeu est petit ; sur dix fois plus, le pilote tombe.

A.8 Étape 8 : conclure, remettre d'aplomb, versionner

Labo : `Labo_07_DE.md`, étape 8.

Notions : section 8, quand une machine ne suffit plus.

- 8.1. Sur ce volume, une machine gagne de deux ordres de grandeur. Le démarrage du conteneur, une vingtaine de secondes, s'ajoute à chaque lancement de Spark.
- 8.3 et 8.4. Le Cours 08 compte sur l'état du Cours 06 : 1 989 fichiers, un historien de 182 jours et 550 087 lignes. Chez l'auteur, 85 s de génération, puis 10,5 s de bronze. Le générateur est déterministe : les mois déjà livrés reviennent identiques au bit près, et la couche `RAW` n'a pas à se refaire. Le bronze de l'historien garde ses 821 dossiers `jour=`, dont 639 vides : le `OVERWRITE` du script du Cours 02 efface les fichiers d'un dossier, pas le dossier. Un dossier vide n'a aucun `*.parquet`, et les lectures des cours suivants l'ignorent.
- 8.6. `*.sh text eol=lf` dit à Git de garder les fins de ligne d'Unix dans tout script shell, même sur un poste Windows.
- 8.7. Le dossier `comparaison/` n'apparaît jamais dans `git status` : il vivait dans `Projet_Boreal`, hors du dépôt, comme tout ce que le pipeline produit.

En entreprise, les mêmes gestes changent d'outil :

© 2026 iAlexMG. Tous droits réservés.

Geste du labo	En entreprise
Spark dans un conteneur à usage unique (3)	une grappe gérée : Databricks, Amazon EMR, Google Dataproc, Azure Synapse ; le labo en montre l'équivalent Databricks dans son annexe
Le lanceur `spark.sh` (3.2)	`spark-submit` vers la grappe, lancé par l'orchestrateur, comme une tâche du Cours 06
La mesure contre une machine (1, 2, 8)	la même question avant tout achat de plateforme, posée avec le vrai volume et le vrai délai
Le PySpark du labo (5 à 7)	Snowpark sur Snowflake, une API de DataFrame dont le code ressemble au PySpark, traduite en SQL par l'entrepôt : la plateforme appartient à la formation Snowflake
Les fins de ligne (3.4, 8.6)	le même `.gitattributes` dans tout dépôt partagé entre Windows et Linux

© 2026 iAlexMG. Tous droits réservés.